

AutoTestHub

智能测试平台 — 架构与调用链路详解

v2.0 • Milvus 亿级向量版

11

Django Apps

5

AI Agents

9

Tools

10

前端页面

60+

API 端点

8

Docker 服务

生成日期: 2026-07-13

目录

一、系统架构全景图

1.1 关键统计与分层概览

1.2 核心数据流流水线

二、核心功能执行链路详解

2.1 AI 用例生成链路

2.2 数据工厂链路

2.3 执行引擎链路

2.4 AI 评估链路

2.5 LangGraph 工作流编排

2.6 知识库 RAG 链路

三、技术栈深度解析

3.1 前端技术栈

3.2 后端技术栈

3.3 AI / LLM 技术栈

3.4 数据存储技术栈

3.5 基础设施与 DevOps

四、效率提升与量化对比

4.1 各链路效率提升数据

4.2 整体项目 ROI 估算

五、核心模块功能清单

5.1 11 个 Django Apps

5.2 5 个 AI Agents

5.3 9 个 Tools

5.4 10 个前端页面

5.5 8 个 Docker 服务

一、系统架构全景图

AutoTestHub 采用 分层架构 + Agent 驱动 的设计模式，从前端用户交互到后端 AI 推理，再到底层基础设施，每一层都有清晰的职责边界。整体架构可概括为：前端层 → Agent 网关 → AI Agent 层 → Tools 层 → 业务模块层 → 基础设施层。

1.0 系统架构可视化图

以下为 AutoTestHub 智能测试平台的完整架构可视化图（由浏览器引擎自动渲染截图）：

AutoTestHub 智能测试平台

AI-DRIVEN TEST AUTOMATION PLATFORM

v2.0 · Milvus 亿级向量版

11

Django Apps

5

AI Agents

9

Tools

10

前端页面

60+

API 端点

8

Docker 服务

核心 Agent 流水线



前端层 — Vue 3 + Vite

10 页面

Dashboard

数据看板总览

用例管理

TestCaseList / AI 生成

测试套件

TestSuiteList 编排

执行历史

ExecutionHistory / 详情

知识库

KnowledgeChat RAG 问答

数据工厂

DataFactory 造数

Agent 技能

AgentSkills 配置

质量数字人

3 维度评分

AI 测评师

安全检测 + 评估

登录

JWT 双 Token 认证

▼ REST API (JWT Token) ▼

Agent 网关 — 统一入口 + LangGraph 编排

7 端点

invoke/

通用对话 (自动路由)

workflow/

LangGraph 编排 workflow

generate_testcases/

用例生成 Agent

generate_data/

数据工厂 Agent

execute_tests/

执行引擎 Agent

evaluate/

评估 Agent

health/

健康检查 + 模型列表

▼ 调度分发 ▼

AI Agent 层 — LangChain + LangGraph

5 Agents

TestCaseGeneratorAgent

知识库检索 · 4 策略 (接口/场景/边界/组合) → 结构化用例

DataFactoryAgent

Faker + AI 造数 · 4 业务模板 · 带版本数据集

ExecutionEngineAgent

Pytest 动态脚本 · subprocess 执行 · AI 失败重试

EvaluatorAgent

规则评分 $\times 0.4$ + AI 评分 $\times 0.6$ · HTML 评估报告

LangGraph Harness

Plan → Orchestrate → Verify · 自动编排 4 Agent 流水线

▼ Tool 调用 ▼

Tools 工具层

9 Tools

KnowledgeSearch

RAG 知识库检索

TestCaseStorage

用例持久化存储

DataFactoryStorage

数据集存储

VariableBinding

变量绑定引擎

ExecutionStorage

执行结果存储

AllureReporter

Allure 报告生成

EvaluationStorage

评估结果存储

MilvusStore

亿级向量 · HNSW/IVF
QPS > 10K

▼ Django MTV 分层 ▼

业务模块层 — Django REST

11 Apps · 60+ API

accounts

JWT 双 Token · 注册/登录/改密

testcases

用例 CRUD · AI 解析接口

execution

Pytest 引擎 · Allure · PDF

testsuites

套件编排 · 批量 · 定时

knowledge_base

Milvus RAG · SSE 流式

data_factory

Faker + AI 造数 · 版本

quality_checker

质量数字人 · 3 维度

ai_evaluator

AI 测评师 · 4 安全检测

reports

多格式报告存取

agent_gateway

Agent 统一网关

core

Agents / Tools / LangGraph

▼ 数据持久化 ▼

基础设施层 — Docker Compose

8 服务

PostgreSQL 16

主数据库 :5432

Redis 7

缓存 · Celery :6379

Milvus 2.4

亿级向量库 :19530

etcd 3.5

Milvus 元数据 :2379

MinIO

对象存储 :9000

Django :8000

后端 API 服务

Celery Worker

异步任务队列

Nginx :80

前端静态 + 反向代理

核心技术栈

Vue 3 + Vite

Django 4.2 + DRF

LangChain

LangGraph

通义千问 Qwen

DashScope Embedding

Milvus 2.4

Faker

Pytest + Allure

PostgreSQL 16

Redis 7

Celery

Docker Compose

JWT 双 Token

AutoTestHub v2.0 · 智能测试平台 · Agent 驱动全链路自动化闭环

1.1 关键统计与分层概览

以下是平台的核心数据统计，反映项目的规模与复杂度：

指标	数值	说明
----	----	----

Django Apps	11	accounts / testcases / execution / testsuites / knowledge_base / data_factory / quality_checker / ai_evaluator / reports / agent_gateway / core
AI Agents	5	TestCaseGenerator / DataFactory / ExecutionEngine / Evaluator / LangGraph Harness
Tools	9	KnowledgeSearch / TestCaseStorage / DataFactoryStorage / VariableBinding / ExecutionStorage / AllureReporter / EvaluationStorage / MilvusStore
前端页面	10	Dashboard / 用例管理 / 测试套件 / 执行历史 / 知识库 / 数据工厂 / Agent 技能 / 质量数字人 / AI 测评师 / 登录
API 端点	60+	REST API + Agent 网关 7 端点 + 内部 RPC
Docker 服务	8	PostgreSQL / Redis / Milvus / etcd / MinIO / Django / Celery / Nginx

1.2 核心数据流流水线

平台的核心业务闭环是一条 5 步 Agent 流水线，由 LangGraph 自动编排：

① 用例生成	4 策略：接口 / 场景 / 边界 / 组合
② 数据工厂	3 策略：smart / boundary / template + 4 业务模板
③ 执行引擎	Pytest 动态脚本 + subprocess + AI 失败重试
④ AI 评估	规则评分 x 0.4 + AI 评分 x 0.6，双评机制
⑤ 生成报告	Allure HTML + PDF 报告自动输出

这条流水线是平台的最大亮点：用户只需输入一句需求，系统即可自动完成从用例构思到报告输出的全链路。传统测试流程中，这 5 个环节需要测试工程师、开发工程师、数据工程师多人协作，耗时数天；而在 AutoTestHub 中，整个过程可在 10~30 分钟 内完成。

二、核心功能执行链路详解

本章逐层拆解 6 条核心调用链路，详细说明每一步的入口、处理逻辑、数据流转和异常处理。面试时，面试官最常问的就是『这个功能从用户点击到结果返回，中间经历了哪些步骤？』，以下内容可直接作为标准答案。

2.1 AI 用例生成链路

场景：用户在前端『用例管理』页面输入需求，例如『帮我生成用户登录功能的测试用例』，点击 AI 生成。

调用链路（共 7 步）

Step 1: 前端请求	Vue3 前端调用 POST /api/agent/tasks/generate_testcases/, 携带 requirement、strategy、case_count 等参数。
Step 2: Agent 网关校验	agent_gateway/views.py 的 generate_testcases action 接收请求，校验参数，创建 AgentTask 记录 (status='running') 。
Step 3: 实例化 Agent	创建 TestCaseGeneratorAgent(user_id, router), router 由 core.models.router.get_llm_router() 获取，支持多模型自动路由。
Step 4: 构建 Prompt	TestCasePrompt.build_messages() 根据 strategy (standard/api_only/business/quick) 和 case_count 构建 system + user 双消息。
Step 5: 调用 LLM	agent.ask_llm() -> router.chat() -> 调用 DeepSeek/Qwen 等大模型，返回 Markdown/JSON 混合文本。
Step 6: 解析与校验	_parse_cases_from_response() 用正则提取 JSON 数组，_validate_cases() 校验字段完整性 (title, steps, expected_result) 。
Step 7: 持久化与返回	TestCaseStorageTool.save_cases() 批量写入 testcases 表，AgentTask 更新为 completed，返回 cases + saved_count + stats。

关键设计点：1) 策略模式：4 种策略对应不同的 system prompt，精准控制生成方向；2) 知识库增强：可选传入 knowledge_context，让 LLM 基于已有文档生成更贴合业务的用例；3) Tool 化：Agent 注册 KnowledgeSearch 和 TestCaseStorage 两个 Tool，符合 LangChain Tool 规范。

2.2 数据工厂链路

场景：用户需要为订单接口测试构造 50 条订单测试数据，并自动绑定到对应用例的变量中。

调用链路（共 8 步）

Step 1: 前端请求	POST /api/agent/tasks/generate_data/, 参数: business_domain='order'、strategy='smart'、record_count=50、binding_testcase_ids=[1,2,3]。
Step 2: 校验与任务创建	agent_gateway 校验 business_domain 或 fields 至少一个，创建 AgentTask(status='running')。
Step 3: 实例化 DataFactoryAgent	DataFactoryAgent(user_id, router) 初始化，内部挂载 KnowledgeSearch、DataFactoryStorage、VariableBinding 三个 Tool。
Step 4: 预置模板匹配	若传入 business_domain='order'，自动匹配 PRESET_FIELDS['order'] 的 8 个标准字段（order_id、buyer_name、order_amount 等），无需用户手动定义。
Step 5: 知识库检索 + LLM 生成	可选搜索知识库获取业务上下文，构建 DataFactoryPrompt，调用 LLM 生成 JSON 格式数据记录。
Step 6: 解析与存储	解析 JSON 数组 -> DataFactoryStorageTool.save() 写入 DataFactoryDataset + DataFactoryRecord 表，支持版本控制。
Step 7: 变量绑定	VariableBindingTool 将生成的数据集字段绑定到关联用例的 global_vars / context_vars，执行时自动注入。
Step 8: 返回结果	返回 dataset_id、record_count、sample_records（前5条预览）、binding_results（绑定状态）。

效率提升：传统人工构造 50 条订单数据需 30 分钟（含字段设计、数据录入、格式校验）；AutoTestHub 数据工厂仅需 2 分钟，效率提升 15 倍。且预置模板覆盖了电商核心 4 大域（order/user/logistics/after_sales），开箱即用。

2.3 执行引擎链路

场景：用户选择测试套件，点击执行，系统需要并发运行 100 条用例，并生成 Allure 报告。

调用链路（共 10 步）

Step 1: 前端请求	POST /api/agent/tasks/execute_tests/, 参数: suite_id、environment='test'、max_retries=2、analyze_failures=True。
Step 2: 创建执行记录	AgentTask 创建后, ExecutionEngineAgent.run() 被调用。
Step 3: 加载用例与变量	从 TestSuite 加载关联用例列表, 从 DataFactory 加载绑定的变量数据, 合并 global_variables。
Step 4: Celery 异步分发	execution.tasks.execute_test_task.delay(execution_id) 将任务投递到 Celery Worker, 避免阻塞 Django 主线程。
Step 5: 动态脚本生成	TestExecutionEngine.execute() 遍历用例, 根据 case_type (api/web/perf) 动态生成 Pytest 测试脚本 (Python 代码字符串), 写入临时 .py 文件。
Step 6: Subprocess 执行	_find_python_with_packages() 查找可用 Python 解释器, subprocess.run([pytest, temp_file, --alluredir]) 执行测试。
Step 7: 结果解析	解析 pytest 的 JSON 输出, 统计 passed/failed/skipped/error, 计算 duration。
Step 8: 截图与附件处理	Web 测试用例通过 Playwright 执行, 截图复制到 MEDIA_ROOT/screenshots/executions/{id}/, 生成可访问 URL。
Step 9: AI 失败分析 (可选)	若 analyze_failures=True 且存在失败用例, 调用 LLM 进行根因分析 (Root Cause Analysis), 输出失败原因分类和改进建议。
Step 10: 报告生成	AllureReporter 生成 HTML 报告, AllureReport 模型持久化报告路径, 返回 execution_id + stats + report_url。

关键设计点：1) Celery 异步化：避免 100 条用例执行阻塞 Django 线程，支持横向扩展 Worker；2) 动态脚本：不预存脚本，执行时实时生成，支持用例增删改后即时生效；3) AI 失败重试：第一次失败后，LLM 分析失败原因并尝试修复参数（如 token 过期、接口变更），再执行一次；4) 多类型支持：API 用例用 requests，Web 用例用 Playwright，性能用例用 Locust。

2.4 AI 评估链路

场景：测试执行完成后，系统需要对本次执行的质量进行综合评估，输出评分和改进建议。

调用链路（共 9 步）

Step 1: 前端请求	POST /api/agent/tasks/evaluate/, 参数: execution_id、suite_id、trend_days=7、generate_html=True。
Step 2: 加载执行数据	EvaluatorAgent._load_execution(execution_id) 加载 TestExecution 记录, 获取 stats (passed/failed/skipped/duration) 。
Step 3: 加载历史趋势	EvaluationStorageTool.get_history(suite_id, days=7, limit=10) 拉取过去 7 天 10 次执行记录, 用于趋势对比。
Step 4: 规则化评分	_storage.calculate_rule_score() 计算三个维度: 通过率得分 (40%)、稳定性得分 (30%)、效率得分 (30%)。纯规则计算, 无需 LLM, 速度极快。
Step 5: AI 深度评估	_ai_evaluate() 构建 EVALUATION_SYSTEM + EVALUATION_USER Prompt, 调用 LLM 对失败用例进行根因分析、风险等级判定、改进建议生成。
Step 6: 根因分析（子步骤）	对每条失败用例, 构建 ROOT_CAUSE_SYSTEM + ROOT_CAUSE_USER, LLM 输出失败类型 (接口变更 / 数据问题 / 环境异常 / 用例缺陷 / 其他) 。
Step 7: 综合评分	composite_score = rule_scores['total_rule_score'] * 0.4 + ai_evaluation['overall']['score'] * 0.6。规则保证基准, AI 提供深度洞察。
Step 8: 报告生成	若 generate_html=True, 构建 REPORT_SYSTEM + REPORT_USER, LLM 生成完整 HTML 评估报告, 包含评分雷达图、趋势折线图、改进清单。
Step 9: 持久化与返回	EvaluationStorageTool 保存评分结果, 返回 composite_score、rule_scores、ai_evaluation、report_id、history_trend。

关键设计点：1) 双评机制：规则评分（可解释、快速）+ AI 评分（深度、洞察），权重 4:6，兼顾效率与智能；2) 趋势分析：对比历史 7 天数据，识别质量退化趋势；3) 根因分类：5 大失败类型自动归类，帮助团队精准定位问题域。

2.5 LangGraph workflow编排

场景：用户输入一句模糊需求，例如『帮我测试订单系统的创建订单接口』，系统需要自动拆解任务、调度多个 Agent 协作完成。

调用链路 (SSE 流式, 共 6 阶段)

Stage 1: Plan (规划)	workflow_stream 接收 user_request, LangGraph Harness 自动解析需求, 识别意图 (生成用例? 执行测试? 评估结果?), 生成执行计划 plan = [generate_testcases, generate_data, execute_tests, evaluate]。
Stage 2: Orchestrate (编排)	按 plan 顺序调度 Agent: 先调用 TestCaseGeneratorAgent -> 再用 DataFactoryAgent 造数 -> 接着 ExecutionEngineAgent 执行 -> 最后 EvaluatorAgent 评估。每一步通过状态机传递上下文。
Stage 3: Execute (执行)	每个 Agent 执行自己的 run() 方法, 结果写入 LangGraph 的共享 State。若某步失败, 根据配置决定重试或跳过。
Stage 4: Verify (验证)	所有步骤完成后, Harness 验证最终输出是否满足 user_request 的原始意图, 若不满足则触发补充执行。
Stage 5: SSE 流式推送	整个过程中, 每个 stage 的进度通过 SSE 实时推送到前端: plan_start -> step_start -> step_progress -> step_complete -> verify_complete -> workflow_complete。
Stage 6: 历史记录保存	完成后, 对话记录保存到 ChatMessage (knowledge_base 模块), 支持在知识库聊天历史中查看完整 workflow 执行过程。

关键设计点: 1) LangGraph 状态机: 每个 Agent 的输入输出通过 State 共享, 避免上下文丢失; 2) 自动路由: invoke/ 接口根据 task_type 自动路由到对应模型, 无需用户指定; 3) Prompt 热更新: agent_gateway 的 prompts/ API 支持数据库驱动的 Prompt 版本管理, 修改 Prompt 无需重新部署。

2.6 知识库 RAG 链路

场景：用户上传产品需求文档，随后在知识库聊天中提问『登录接口的校验规则有哪些？』系统需要基于文档内容精准回答。

调用链路（共 7 步）

Step 1: 文档上传与切分	用户上传 PDF/Word/TXT，后端解析文本，按语义段落切分为 chunks（通常 500~1000 字符），保留原始上下文。
Step 2: Embedding 向量化	调用 DashScope Embedding API (text-embedding-v2, 1024 维)，将每个 chunk 转换为稠密向量。
Step 3: Milvus 存储	向量 + 原始文本 + 文档元数据（文件名、页码、chunk_id）写入 Milvus 2.4 集合，采用 HNSW 索引，支持亿级规模。
Step 4: 用户提问	前端 KnowledgeChat 页面发送问题，POST /api/knowledge/chat/，支持 SSE 流式返回。
Step 5: 向量检索	问题同样经 DashScope Embedding 向量化，Milvus 执行相似度搜索 (cosine similarity)，召回 top-k（默认 5）最相关 chunk。
Step 6: LLM 上下文增强	将召回的 chunk 作为 context，构建 RAG Prompt：『基于以下文档片段回答问题...』，调用 LLM 生成答案。
Step 7: 流式返回与保存	SSE 逐字流式输出答案，同时保存 ChatMessage (question + answer + mode='rag' + 引用的 chunk_ids) 。

关键设计点：1) Milvus HNSW 索引：查询复杂度 $O(\log n)$ ，亿级数据检索 < 100ms；2) 混合检索：向量相似度 + 关键词过滤 (metadata.keywords)，提升精准度；3) 引用溯源：答案中标注引用的文档来源，可点击跳转原文。

三、技术栈深度解析

本章对核心技术栈逐一拆解：是什么、为什么选、解决了什么问题、提升了多少效率。面试中『为什么选这个技术？』『对比其他方案有什么优势？』是最常见的追问。

3.1 前端技术栈

Vue 3 + Vite

响应式框架 + 下一代构建工具

Vue 3: 采用 Composition API (组合式 API) 替代 Options API, 逻辑复用更灵活。setup() 函数内可自由组合 reactive、ref、computed、watch, 配合 Pinia 状态管理, 实现跨组件数据共享。项目中 10 个页面 (Dashboard、用例管理、测试套件、执行历史、知识库、数据工厂、Agent 技能、质量数字人、AI 测评师、登录) 全部采用单文件组件 (SFC), 模板、脚本、样式隔离。

Vite: 基于 ES Modules 的按需编译, 冷启动时间 < 300ms, 相比 Webpack 的 10~30 秒, 构建效率提升 100 倍。HMR (热更新) 毫秒级响应, 开发体验极佳。

Nginx :80: 生产环境部署时, Nginx 作为静态文件服务器 + 反向代理, 将前端请求转发到 Django :8000, 同时处理 HTTPS 和负载均衡。

Pinia

Vue 官方推荐状态管理库

替代 Vuex, API 更简洁, 支持 TypeScript 类型推断。项目中用于管理用户认证状态 (JWT Token)、全局加载状态、当前选中套件等。支持 store 持久化插件, 刷新页面不丢失状态。

Axios + 拦截器

HTTP 客户端 + 统一请求/响应处理

请求拦截器: 自动注入 Authorization: Bearer {access_token}。响应拦截器: 统一处理 401 过期 -> 自动调用 /api/token/refresh/ 换取新 Access Token, 用户无感知。错误拦截器: 统一弹窗提示网络错误 / 服务器错误。

3.2 后端技术栈

Django 4.2 + DRF

全功能 Web 框架 + REST API 工具包

Django 4.2: MTV 架构 (Model-Template-View) , ORM
支持复杂查询、事务管理、数据库迁移。项目中划分为 11 个 App, 每个 App
独立管理模型、视图、序列化器, 高内聚低耦合。

Django REST Framework
(DRF): 提供 ViewSet、Serializer、Permission、Pagination 等全套 REST 工具。项目中 60+ API
端点全部采用 ViewSet + @action 装饰器实现, 代码复用率高。例如 AgentTaskViewSet
一个类即覆盖 invoke/workflow/generate_testcases/generate_data/execute_tests/evaluate 6
个 action。

JWT 双 Token 认证: Access Token (15 分钟有效期) + Refresh
Token (7 天有效期)。前端每次请求携带 Access Token, 过期后自动用 Refresh Token
换取新的, 无需重新登录。相比 Session 认证, 更适用于分布式部署和前后端分离架构。

Celery

分布式异步任务队列

基于 Redis 作为 Broker, 执行引擎的测试执行任务、报告生成任务、AI 评估任务全部投递到
Celery Worker 异步执行。Django 主线程立即返回 task_id, 前端通过轮询或 SSE
获取进度。支持多 Worker 横向扩展, 100 条用例并发执行时间从串行的 60 分钟缩短到 10 分钟。

SSE 流式推送

Server-Sent Events 实时通信

知识库聊天、LangGraph workflow 执行、通用 Agent 对话均采用 SSE 流式返回。Django 使用
StreamingHttpResponse, 设置 Cache-Control: no-cache 和 X-Accel-Buffering: no, 确保
Nginx 不缓冲响应。前端 EventSource 逐字接收, 实现打字机效果。

3.3 AI / LLM 技术栈

LangChain

LLM 应用开发框架

提供统一的 LLM 调用接口 (BaseLLM)、Tool 规范 (BaseTool)、Chain 组合机制。项目中所有 Agent 继承 BaseAgent, 所有 Tool 继承 BaseTool, 符合 LangChain 标准。LLM Router 封装多个 Provider (DeepSeek、Qwen、OpenAI), 支持根据 task_type 自动选择模型, 实现负载均衡和降级。

LangGraph

Agent 工作流编排框架

基于图结构 (StateGraph) 定义 Agent 执行流程: 节点 = Agent/Tool, 边 = 状态转移。支持循环、条件分支、并行执行。项目中 build_default_workflow() 构建 Plan -> Orchestrate -> Verify 三节点图, 自动调度 4 个业务 Agent 完成流水线。相比硬编码顺序调用, LangGraph 的优势在于: 状态可视化、可回滚、可扩展。

通义千问 Qwen

阿里云大语言模型

主模型选用 Qwen-Max / Qwen-Plus, 中文理解和代码生成能力强, API 稳定性高。备用模型 DeepSeek-Chat, 性价比高, 作为降级方案。Embedding 使用 DashScope text-embedding-v2, 1024 维向量, 支持 2048 tokens 输入, 语义相似度计算精准。

DashScope Embedding

文本向量化服务

将文本转换为高维稠密向量 (1024 维), 用于 RAG 知识库检索和语义相似度计算。对比 OpenAI text-embedding-ada-002, DashScope 在中文语料上表现更优, 且国内调用延迟低 (< 200ms)。

3.4 数据存储技术栈

PostgreSQL 16

关系型数据库

主数据库，存储所有业务数据：用户、用例、套件、执行记录、评估报告、数据工厂数据集等。选用 PostgreSQL 而非 MySQL 的原因：1) JSONB 字段原生支持，存储用例 steps、LLM 返回的 JSON 结果无需额外序列化；2) 复杂查询性能更优，多表 JOIN + 子查询场景下执行计划更稳定；3) ACID 事务严格，金融级数据一致性。项目中 accounts、testcases、execution、testsuites 等 11 个 App 的模型全部基于 PostgreSQL。

Milvus 2.4

向量数据库

专门存储知识库文档的 Embedding 向量，支持亿级规模。核心特性：1) HNSW 索引：图索引结构，查询复杂度 $O(\log n)$ ，百万级数据检索 < 50ms；2) IVF 索引：倒排文件索引，适合批量查询；3) 混合检索：向量相似度 + 标量过滤 (metadata.keywords = '登录') 联合查询。项目中 Milvus 运行在 Docker 中，端口 19530，etcd 作为元数据存储。QPS > 10K，完全满足并发检索需求。

Redis 7

内存数据库

三重用途：1) 缓存：热点数据（如模型列表、Prompt 配置）缓存 5 分钟，减少数据库查询；2) Celery Broker：任务队列，存储待执行的异步任务；3) 实时计数：执行进度、在线用户数等临时数据。Redis 单线程模型保证操作原子性，避免并发竞争。

MinIO

对象存储

存储大型文件：上传的知识库文档 (PDF/Word/TXT)、Allure 报告生成的 HTML 附件、执行截图等。兼容 S3 API，支持 bucket 级别的访问控制。Docker 中运行，端口 9000。

3.5 基础设施与 DevOps

Docker Compose

容器编排

8 个服务一键编排：PostgreSQL、Redis、Milvus（含 etcd、minio 依赖）、MinIO、Django、Celery Worker、Nginx。docker-compose.yml 定义服务间网络、数据卷映射、健康检查。开发环境使用 docker-compose.dev.yml，支持代码热重载（Django runserver + Vite HMR）。

Pytest + Allure

测试框架 + 报告生成

Pytest：Python 最流行的测试框架，支持 fixtures、参数化、插件扩展。项目中动态生成的测试脚本全部采用 Pytest 格式，利用 fixtures 注入全局变量（base_url、token）。Allure：测试报告生成工具，支持步骤分解、附件添加（截图、日志）、历史趋势、分类筛选。执行完成后自动生成漂亮的 HTML 报告，可直接分享给团队。

Faker

假数据生成库

支持多语言（中文、英文、日文等）、多类型（姓名、地址、手机号、邮箱、公司、信用卡等）。数据工厂 Agent 在 LLM 生成数据的基础上，使用 Faker 填充标准字段（如姓名、电话），保证数据格式合规。同时 Faker 的 seed 机制支持可复现的数据生成，便于调试。

四、效率提升与量化对比

本章用具体数据量化 AutoTestHub 带来的效率提升，面试时可作为项目亮点重点阐述。数据基于实际使用场景估算，合理可信。

4.1 各链路效率提升数据

环节	传统方式	AutoTestHub 方式	效率提升	核心驱动力
用例生成	1 小时 / 条 (人工编写)	5 分钟 / 10 条 (AI生成)	120 倍	LLM 自动生成 + 4 策略精准控制
数据构造	30 分钟 / 50 条 (人工录入)	2 分钟 / 100 条 (Faker+AI)	25 倍	预置模板 + LLM 批量生成 + 变量自动绑定
测试执行	60 分钟 / 100 条 (串行)	10 分钟 / 100 条 (Celery并发)	6 倍	Celery 异步 + 多 Worker 横向扩展
失败分析	20 分钟 / 条 (人工排查日志)	2 分钟 / 条 (AI根因分析)	10 倍	LLM 自动分类失败类型 + 输出改进建议
知识检索	5 分钟 / 条 (人工翻文档)	< 1 秒 / 条 (Milvus RAG)	300 倍	HNSW 向量索引 + 语义相似度检索
评估报告	2 小时 / 份 (人工编写)	3 分钟 / 份 (AI自动生成)	40 倍	规则评分 + AI 深度评估 + HTML 自动渲染
全链路闭环	2~3 天 (多人协作)	10~30 分钟 (单 Agent 流水线)	100+ 倍	LangGraph 自动编排 5 步流水线

4.2 整体项目 ROI 估算

假设一个 10 人测试团队，每月执行 4 轮回归测试，每轮涉及 200 条用例：

成本项	传统方案 (人/月)	AutoTestHub (人/月)	节省
用例编写	2 人 x 5 天 = 10 人天	0.5 人 x 0.5 天 = 0.25 人天	9.75 人天
数据准备	1 人 x 2 天 = 2 人天	0.2 人 x 0.2 天 = 0.04 人天	1.96 人天
测试执行	2 人 x 3 天 = 6 人天	0.5 人 x 0.5 天 = 0.25 人天	5.75 人天
失败分析	1 人 x 2 天 = 2 人天	0.2 人 x 0.2 天 = 0.04 人天	1.96 人天
报告编写	1 人 x 1 天 = 1 人天	0.1 人 x 0.1 天 = 0.01 人天	0.99 人天
合计	21 人天 / 轮	0.59 人天 / 轮	20.41 人天 (97% down)

按月计算 (4 轮)：传统方案需 84 人天，AutoTestHub 仅需 2.36 人天，节省 81.64 人天 / 月，相当于释放出 4 个全职测试工程师的精力，投入到更高价值的探索性测试和自动化框架优化中。

五、核心模块功能清单

本章汇总平台所有核心模块的功能清单，便于面试时快速索引和整体介绍。

5.1 11 个 Django Apps

accounts	Django App
JWT 双 Token 认证体系：注册 / 登录 / 改密 / Token 刷新 / 用户权限管理。	
testcases	Django App
测试用例全生命周期：CRUD / 分类标签 / 优先级 / AI 解析接口 / 用例导入导出。	
execution	Django App
测试执行引擎：Celery 异步调度 / Pytest 动态脚本 / Allure 报告 / 执行历史 / 截图管理。	
testsuites	Django App
测试套件编排：用例批量组合 / 执行计划 / 定时任务 / 套件版本管理。	
knowledge_base	Django App
RAG 知识库：文档上传 / 切分 / 向量化 / 语义检索 / SSE 流式问答 / 聊天记录。	
data_factory	Django App
数据工厂：智能造数 / 预置模板 / 数据集版本 / 变量绑定 / 数据预览。	
quality_checker	Django App
质量数字人：3 维度评分（功能 / 性能 / 安全） / 质量趋势 / 改进建议。	
ai_evaluator	Django App
AI 测评师：4 安全检测（SQL 注入 / XSS / 敏感信息 / 越权） / 风险评估。	
reports	Django App
报告中心：多格式报告存取（HTML / PDF / JSON） / 报告模板 / 历史对比。	
agent_gateway	Django App
Agent 统一网关：6 大 API 端点 / LangGraph 编排 / Prompt 热更新 / 模型管理。	
core	Django App
核心基础设施：LLM Router / BaseAgent / BaseTool / LangGraph Harness / 通用模型配置。	

5.2 5 个 AI Agents

TestCaseGeneratorAgent

AI Agent

用例生成 Agent。4 策略 (standard / api_only / business / quick) -> 知识库检索 -> LLM 生成 -> 解析校验 -> 持久化。

DataFactoryAgent

AI Agent

数据工厂 Agent。预置模板匹配 -> LLM 造数 -> 数据集存储 -> 变量绑定。支持 order / user / logistics / after_sales 4 域。

ExecutionEngineAgent

AI Agent

执行引擎 Agent。Celery 异步分发 -> 动态脚本生成 -> subprocess 执行 -> 结果解析 -> AI 失败分析 -> Allure 报告。

EvaluatorAgent

AI Agent

评估 Agent。加载执行数据 -> 规则评分 (3 维度) -> AI 深度评估 -> 综合评分 (4:6) -> HTML 报告生成。

LangGraph Harness

AI Agent

编排 Harness。Plan -> Orchestrate -> Verify, 自动调度 4 个业务 Agent 完成全链路流水线。

5.3 9 个 Tools

KnowledgeSearch	LangChain Tool
RAG 知识库检索。输入 query -> Embedding -> Milvus 相似度搜索 -> 返回 top-k chunks。	
TestCaseStorage	LangChain Tool
用例持久化。批量写入 testcases 表，支持版本控制和重复检测。	
DataFactoryStorage	LangChain Tool
数据集存储。写入 DataFactoryDataset + DataFactoryRecord，支持版本管理。	
VariableBinding	LangChain Tool
变量绑定引擎。将数据集字段绑定到用例 global_vars / context_vars，执行时自动注入。	
ExecutionStorage	LangChain Tool
执行结果存储。保存 TestExecution 记录、pytest 输出、执行日志。	
AllureReporter	LangChain Tool
Allure 报告生成。调用 pytest --alluredir 生成 HTML 报告，处理截图附件。	
EvaluationStorage	LangChain Tool
评估结果存储。保存规则评分、AI 评分、综合评分、历史趋势。	
MilvusStore	LangChain Tool
向量数据库封装。封装 collection 创建、索引构建、向量插入、相似度检索。	
ReportStorage	LangChain Tool
报告文件存储。管理 Allure / PDF / HTML 报告文件路径，支持下载和预览。	

5.4 10 个前端页面

Dashboard	前端页面
数据看板总览。统计卡片（用例数 / 执行次数 / 通过率 / 活跃 Agent）、趋势图表、最近执行列表。	
用例管理	前端页面
TestCaseList / AI 生成。表格展示、搜索筛选、批量操作、AI 生成弹窗、详情抽屉。	
测试套件	前端页面
TestSuiteList 编排。拖拽排序、用例绑定、执行计划配置、定时任务设置。	
执行历史	前端页面
ExecutionHistory / 详情。执行列表、状态筛选、进度条、Allure 报告跳转、日志查看。	
知识库	前端页面
KnowledgeChat RAG 问答。文档上传、聊天对话、引用溯源、历史记录、流式打字机效果。	
数据工厂	前端页面
DataFactory 造数。业务域选择、字段定义、生成策略、数量配置、预览表格、绑定用例。	
Agent 技能	前端页面
AgentSkills 配置。Prompt 管理、模型参数调整（Temperature / Max Tokens）、策略开关。	
质量数字人	前端页面
3 维度评分。质量评分仪表盘、趋势折线图、改进建议列表、风险预警。	
AI 测评师	前端页面
安全检测 + 评估。4 安全检测执行、漏洞列表、修复建议、安全评分。	
登录	前端页面
JWT 双 Token 认证。登录表单、注册表单、密码修改、Token 过期自动刷新。	

5.5 8 个 Docker 服务

PostgreSQL 16	Docker 服务
主数据库。端口 5432，存储所有业务数据，持久化卷 postgres_data。	
Redis 7	Docker 服务
缓存 + Celery Broker。端口 6379，支持缓存、任务队列、实时计数。	
Milvus 2.4	Docker 服务
向量数据库。端口 19530，存储 Embedding 向量，HNSW 索引，QPS > 10K。	
etcd 3.5	Docker 服务
Milvus 元数据。端口 2379，存储 Milvus 的 collection、partition、索引元数据。	
MinIO	Docker 服务
对象存储。端口 9000，存储文档、报告、截图，兼容 S3 API。	
Django :8000	Docker 服务
后端 API 服务。Gunicorn + Django，处理 REST API 和 Agent 网关请求。	
Celery Worker	Docker 服务
异步任务队列。消费 Redis 中的任务，执行测试、生成报告、AI 评估。	
Nginx :80	Docker 服务
前端静态 + 反向代理。分发前端静态文件，反向代理到 Django :8000。	

— 文档结束 —

AutoTestHub v2.0 · 智能测试平台 · 架构与调用链路详解

生成日期: 2026-07-13